

Desafio Scrum - DIO

1. Por que a abordagem ágil foi utilizada em vez da tradicional?

O artigo constrói essa justificativa por contraste com o Waterfall, não como uma decisão explícita de projeto. Os pontos que ele levanta:

- **QA entrando tarde demais.** No Waterfall, o QA só é envolvido no final do projeto, quando toda a codificação já foi concluída, recebendo um documento de requisitos e o código pronto para verificar se a aplicação faz exatamente o que o documento descreve. Isso significa que problemas de entendimento do requisito só aparecem quando já é caro corrigi-los.
- **O ciclo "build-test-fix".** A autora aponta que esse ciclo repetido do Waterfall gera muito trabalho extra e normalmente desperdiça bastante tempo — o time constrói, só depois testa, descobre o problema, e refaz.
- **Trabalho síncrono vs. assíncrono.** Diferente das atividades síncronas de um projeto Waterfall tradicional, o Scrum espera que as atividades de desenvolvimento sejam realizadas na ordem em que são necessárias — ou seja, de forma assíncrona, permitindo que QA, dev e BA trabalhem em paralelo dentro do mesmo sprint, e não em fases estanques.
- **Feedback contínuo em ciclos curtos.** Entregar em iterações de 2 a 4 semanas dá visibilidade de problemas de requisito, qualidade ou escopo muito antes do que esperar o projeto inteiro terminar.

Em resumo: a escolha pelo ágil resolve um problema estrutural do Waterfall — a validação (teste) fica desacoplada da construção por meses, e qualquer erro de entendimento só aparece no fim, quando é mais caro corrigir.

2. Por que o Scrum foi utilizado?

Aqui o artigo é mais descritivo do framework do que argumentativo sobre a escolha — ele não compara Scrum com Kanban, XP ou outros frameworks ágeis, apenas parte do pressuposto de que o Scrum é o framework em uso e explica como o QA opera dentro dele. As características que o texto usa para justificar implicitamente essa escolha são as duas que abrem o artigo:

- **Times multifuncionais** (cross-functional): reúnem todas as competências necessárias — dev, QA, BA — em um único time, eliminando a "equipe de teste separada" que caracteriza o Waterfall.
- **Times auto-organizáveis:** o próprio time decide como o trabalho será feito, o que dá ao QA espaço para atuar como proxy Product Owner, ajudar na estimativa, liderar a estratégia de teste (já que não existe um "líder de QA" formal no Scrum) etc.

Some ceremonies and structures do the rest of the work: Sprint/Release Planning (onde QA participa da estimativa das histórias), Sprint Review (onde QA frequentemente conduz a demo), e a Definition of Done (que QA monitora). É essa combinação de papéis compartilhados + cerimônias curtas e recorrentes que permite ao QA deixar de ser "quem testa no final" para virar parte constante do fluxo de trabalho.

3. O resultado final era inovador? Por quê?

Esta é a pergunta em que o enquadramento do artigo mais importa. Não há um produto de software descrito, nem métricas de resultado (taxa de defeitos, velocidade, satisfação do cliente) — então, no sentido literal de "o produto entregue era inovador", o artigo simplesmente não permite responder.

O que dá para avaliar como resultado é outra coisa: a **redefinição do papel do QA** dentro do time. A autora descreve o QA fazendo muito mais do que testar — atuando como proxy Product Owner, participando de estimativas, ajudando a definir critérios de aceite, conduzindo demos, e convergindo com o papel do Business Analyst. Isso é uma ruptura real com a imagem tradicional do QA como "gatekeeper" isolado que só aparece no fim.

Dito isso, eu seria cauteloso em chamar isso de inovador no sentido forte da palavra: já em 2012, quando o artigo foi publicado, esses princípios (QA integrado desde o início, "qualidade é responsabilidade de todo o time", testes automatizados como parte do pipeline) já circulavam na comunidade ágil — o livro *Agile Testing* de Lisa Crispin e Janet Gregory, por exemplo, é de 2009. Então o valor do artigo está menos em propor algo novo para a indústria e mais em documentar, de forma honesta e pessoal, como essa mudança de papel se sentiu na prática para quem vinha de uma cultura mais tradicional — o que era razoavelmente inovador *para as organizações* que ainda estavam migrando do Waterfall naquele momento, mesmo sem ser inovador para a comunidade ágil como um todo.

4. O que eu faria diferente?

Algumas lacunas que eu apontaria se estivesse revisando esse relato:

- **Nenhum dado quantitativo.** Tudo é anedótico — sem números de redução de defeitos, tempo de ciclo, velocidade do time antes/depois. Um relato mais forte traria pelo menos indicadores relativos (mesmo que aproximados).
- **Sobrecarga do QA sem discutir o custo disso.** A autora empilha muitos papéis no QA (proxy PO, "consciência de qualidade" do time, estrategista de teste, apresentador de demo, par de programação com devs) sem discutir limites de capacidade. Ela chega a admitir o risco no final — nem o papel de tester nem o de analista de negócio recebem tanta atenção quanto receberiam com alguém dedicado exclusivamente a isso — mas não propõe nenhuma mitigação.
- **Conflito de interesse no papel de proxy PO.** Se o QA às vezes escreve os critérios de aceite como proxy PO e depois testa a própria história contra esses critérios, perde-se uma checagem independente. Eu sugeriria um rodízio da função ou um gatilho claro para quando uma decisão exige o PO real.
- **Ausência de fricções e erros.** O texto lê como um "melhor dos casos" — sem menção a sprints que deram errado, discordâncias com o PO, dívida técnica gerada por automação apressada. Um relato mais equilibrado ganharia credibilidade incluindo isso.
- **Pouco detalhe sobre a automação em si.** A automação é apontada como essencial, mas não há nada sobre pirâmide de testes, o que foi priorizado para automatizar, ou como testes instáveis (flaky) foram tratados.

5. Outros tópicos interessantes

- **Definition of Done como artefato leve, mas central.** A lista de critérios Código completo, teste unitário completo, teste funcional/UI completo, aprovado pelo Product Owner é simples, mas o artigo destaca que ela não é estática — evolui com o time, e pode existir tanto no nível de sprint quanto de release. Vale comparar DoD com "critério de aceite": o primeiro é genérico para qualquer história, o segundo é específico por história.
- **Convergência entre QA e Business Analyst.** O artigo nota que os dois papéis passam a compartilhar responsabilidades (analisar histórias, refinar critérios de aceite), o que conecta com a discussão mais ampla sobre profissionais "T-shaped" em times ágeis pequenos — versátil, mas com o risco de diluição que a própria autora reconhece.
- **"Shift-left" testing.** Embora o artigo não use esse termo (ele só aparece com mais força na literatura um pouco depois), tudo que ele descreve — QA nas estimativas, QA definindo critérios de aceite antes do código existir — é exatamente essa prática: antecipar a qualidade para o início do ciclo, em vez de tratá-la como etapa final.
- **O contexto temporal do artigo.** É interessante ler o texto sabendo que, em 2012, a pergunta "como um testador contribui antes de existir código?" ainda era vista como legítima e intrigante por muitas organizações. Hoje, com testes automatizados totalmente integrados a pipelines de CI/CD e práticas de TDD/BDD consolidadas, essa pergunta já está bastante respondida pela indústria — o que dá uma boa medida de como a profissão mudou nesses ~14 anos.